

Proyecto I

Programación Vectorial en Ensamblador x86-64
(NASM/Linux)

Juan Pablo Solano Solano, 2020425831, Alejandro Vasquez, 2020007275

Índice

1. Introducción	3
1.1. Objetivos del trabajo	3
2. Entorno de prueba	3
3. Implementación escalar	4
3.1. sum_array escalar	5
3.2. compute_stats escalar	6
3.3. normalize_array escalar	8
4. Implementacion vectorial AVX2	10
4.1. sum_array vectorial	11
4.1.1. Inicializacion y calculo del limite vectorial	11
4.1.2. Bucle vectorial de suma	12
4.1.3. Reduccion horizontal del acumulador vectorial	13
4.1.4. Bucle escalar para el remanente	13
4.2. compute_stats vectorial	13

4.2.1.	Preservacion de argumentos y calculo de la media	14
4.2.2.	Inicializacion de acumuladores vectoriales	14
4.2.3.	Bucle vectorial de varianza, minimo y maximo	15
4.2.4.	Reduccion horizontal de la varianza	16
4.2.5.	Reduccion horizontal de minimo y maximo	16
4.2.6.	Bucle escalar para el remanente y almacenamiento de resultados	17
4.3.	normalize_array vectorial	17
4.3.1.	Inicializacion y deteccion del caso stddev == 0	18
4.3.2.	Preparacion del camino vectorial	18
4.3.3.	Bucle vectorial de normalizacion	18
4.3.4.	Bucle escalar para el remanente	19
4.3.5.	Camino de copia cuando stddev == 0	19
5.	Casos de prueba y verificacion de correctud	20
6.	Resultados de Rendimiento	21
7.	Evidencia de la sesión de GDB	23
7.1.	Evidencia de GDB Escalar	23
7.1.1.	GDB en sum_array Escalar	23
7.1.2.	GDB en compute_stats Escalar	26
7.1.3.	GDB en Normalize_array Escalar	28
7.2.	Evidencia de GDB vectorial	30
8.	Conclusiones	31
9.	Limitaciones observadas y trabajo futuro	32

1. Introducción

Para la ejecución de cualquier programa uno de los elementos más relevantes por los que cada desarrollador debe preocuparse es la eficiencia y el tiempo de ejecución. En el mundo del procesamiento de datos existen muchas arquitecturas y lenguajes distintos a los que a cada uno se le atribuye una forma distinta de procesar información. Uno de los lenguajes de programación más usados para este fin es el X86-64. En el presente trabajo se pretende desarrollar la solución de un caso planteado en el procesamiento de datos usando X86, con el fin de comprobar al menos 2 tipos distintos resupuestas al mismo problema, comparar resultados, verificar su eficiencia y cual es la diferencia entre cada uno [1].

1.1. Objetivos del trabajo

Objetivo general. Implementar y comparar una solución escalar y una solución vectorial AVX2, escritas en ensamblador x86-64, para calcular estadísticos descriptivos de un arreglo de datos float32 y producir su versión normalizada.

Objetivos específicos.

1. Implementar `compute_stats` y `normalize_array` en ambas versiones, respetando la convención de llamada System V AMD64 ABI entre el driver en C y las funciones NASM.
2. Procesar ocho elementos por iteración en los bucles AVX2, utilizando memoria alineada a 32 bytes y un bucle escalar para los elementos que no completan un bloque vectorial.
3. Verificar los estadísticos y el arreglo normalizado mediante casos de prueba y una referencia en Python, e inspeccionar con GDB registros y memoria durante la ejecución.
4. Medir los tiempos de ejecución para diferentes tamaños de entrada, calcular el speedup de la versión vectorial y analizar los factores que limitan la aceleración obtenida.

2. Entorno de prueba

Las pruebas se realizaron en Ubuntu 24.04.4 LTS sobre WSL2, con un procesador 12th Gen Intel Core i5-12600K y arquitectura x86_64. El entorno Linux reportó 16 procesadores lógicos disponibles. Se consultó `lscpu` para identificar el modelo de CPU y se revisaron las banderas de `/proc/cpuinfo` para confirmar el soporte de AVX2. Esta última consulta produjo `avx2`, como se muestra en la Figura 1.

```

PRETTY_NAME="Ubuntu 24.04.4 LTS"
6.6.87.2-microsoft-standard-WSL2
Architecture:          x86_64
CPU(s):                16
Model name:            12th Gen Intel(R) Core(TM) i5-12600K
avx2
NASM version 2.16.01
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
GNU gdb (Ubuntu 15.1-1ubuntu1~24.04.1) 15.1

```

Figura 1: Sistema, procesador, confirmación del soporte AVX2 y versiones de las herramientas utilizadas.

Para ensamblar y compilar se utilizaron NASM 2.16.01 y GCC 13.3.0, respectivamente. La inspección de registros y memoria se realizó con GNU GDB 15.1. La identificación de WSL2 también es relevante para interpretar las limitaciones encontradas al intentar medir contadores de hardware.

3. Implementación escalar

Para la implementación escalar se habla del uso de sumas y operaciones de algebraicos lineales, es decir que las operaciones se hacen de una a una. La arquitectura x86 incluye las extensiones de registros XMM0 y YMM0, estos registros introducidos en las extensiones SSE (Streaming SIMD Extensions) y AVX (Advanced Vector Extensions) respectivamente, son registros tipo SIMD (Single Instructions Multiple Data) que permite el manejo de información de manera escalar y vectorial, su constitución se aprecia en la Figura 2

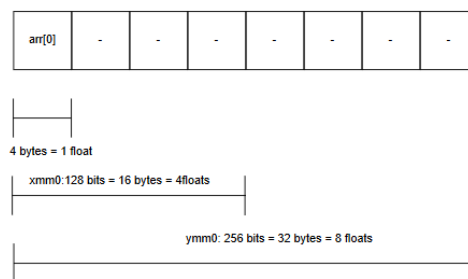


Figura 2: Tamaño de registros XMM0

La solución escalar consiste en tomar como referencia el proceso de la Figura 3 y dividirlo en los módulos correspondientes.

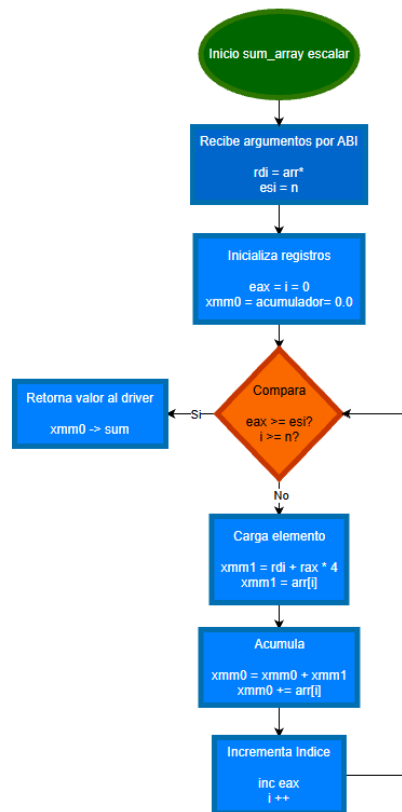


Figura 3: Diagrama de procesamiento Escalar

3.1. sum_array escalar

Para este primer módulo se describe la sección de código de la Figura 4

```

sum_array:
    xor    eax, eax           ; i = 0
    xorps  xmm0, xmm0        ; xmm0 = 0.0 (Acumulador)

.sum_loop:
    cmp    eax, esi          ; i >= n?
    jge    .sum_done
    movss  xmm1, [rdi + rax*4] ; xmm1 = arr[i]
    addss  xmm0, xmm1        ; suma += arr[i]
    inc    eax               ; i++
    jmp    .sum_loop

.sum_done:
    ret
  
```

Figura 4: Sum_Array Escalar

Este módulo se divide en 3 puntos, la definición del sum_array, el sum_loop y el sum_done. En la definición del sum_array se usan los comandos xor y xorps, estos hacen una comparación tipo xor según el tipo de registro que se emplea, esta es una de las maneras más prácticas de poner todo "eax" y "XMM0" en 0. Para este caso XMM0 va a actuar como el acumulador y eax actúa como una constante "i"

El `.sum_loop` es la parte que hace el mayor trabajo. El registro "esi" es el que tiene un valor n que se obtiene del driver. Este valor se compara con el registro `eax` de manera que se sepa si $i \geq n$ (recordar que i es `eax` y comienza en 0 por el `xor`). En el momento en que n sea menor a i la suma está completa y salta al `.sum_done`, de lo contrario se mueve a `XMM1` (otro registro tipo XMM) el primer valor del array de datos que estamos sumando, el primer valor de i es 0 por lo tanto `xmm1 = arr[i]` (siendo `eax = i` y siendo `rax` una forma más completa de `eax`). El valor encontrado en esta posición del array pasa a `xmm1` y seguidamente a `xmm0` y se incrementa `eax` en 1. Esto se repite tantas veces hasta que $eax \geq n$. En ese punto se pasa a `sum_sone` y se completa el proceso ya que `XMM0` posee la suma de todos los valores del array

3.2. compute_stats escalar

Para este segundo módulo se describe la sección de código de las Figuras 5 y 6. Este módulo tiene como objetivo obtener las principales estadísticas del arreglo de datos: la media, la varianza, el valor mínimo y el valor máximo.

```
compute_stats:

    test    esi, esi
    jz      .stats_empty

    push    rbx
    push    rbp
    push    r12
    push    r13
    push    r14
    push    r15

    ; Alinear stack a 16 bytes antes de CALL
    sub     rsp, 8

    mov     rbx, rdi          ; rbx = arr
    mov     r12d, esi         ; r12d = n
    mov     r13, rdx          ; r13 = mean*
    mov     r14, rcx          ; r14 = var*
    mov     r15, r8           ; r15 = min*
    mov     rbp, r9           ; rbp = max*

    mov     rdi, rbx          ; argumento arr
    mov     esi, r12d         ; argumento n
    call    sum_array         ; xmm0 = sum(arr)

    cvtsi2ss xmm4, r12d       ; xmm4 = (float)n

    ; mean = sum / n
    divss   xmm0, xmm4        ; xmm0 = mean

    xor     eax, eax          ; i = 0

    movss   xmm2, [rbx]       ; min = arr[0]
    movss   xmm3, [rbx]       ; max = arr[0]

    xorps   xmm5, xmm5        ; acc_var = 0.0
```

Figura 5: Definición del `Compute_stats`

```

.stats_loop:

    cmp     eax, r12d           ; i >= n?
    jge     .stats_done
    movss   xmm1, [rbx + rax*4] ; xmm1 = arr[i]

    minss   xmm2, xmm1 ; min = min(min, arr[i])
    maxss   xmm3, xmm1 ; max = max(max, arr[i])

    movss   xmm6, xmm1        ; xmm6 = arr[i]
    subss   xmm6, xmm0        ; xmm6 = arr[i] - mean
    mulss   xmm6, xmm6        ; xmm6 = (arr[i] - mean)^2
    addss   xmm5, xmm6        ; acc_var += ...

    inc     eax                ; i++
    jmp     .stats_loop

; -----
; Finalizar estadísticas
; -----

.stats_done:

    divss   xmm5, xmm4        ; var = acc_var / n

    movss   [r13], xmm0       ; *mean = mean
    movss   [r14], xmm5       ; *var = var
    movss   [r15], xmm2       ; *min = min
    movss   [rbp], xmm3       ; *max = max

    add     rsp, 8
    pop     r15
    pop     r14
    pop     r13
    pop     r12
    pop     rbp
    pop     rbx

    ret

.stats_empty:

    xorps   xmm0, xmm0

    movss   [rdx], xmm0       ; mean = 0
    movss   [rcx], xmm0       ; var = 0
    movss   [r0], xmm0        ; min = 0
    movss   [r9], xmm0        ; max = 0

    ret

```

Figura 6: Definición del .stats_loop, .stats_done y .stats_empty

Este módulo nuevamente se divide en 3 puntos, la definición del compute_stats, el ciclo dado por .stats_loop y el stats_done. En la definición del compute_stats primero se pueba si "esi" es 0 ("esi" guarda el valor n, es decir, cantidad de elementos dados por el driver), si es 0 entoces significa que no existen elementos sobre los cuales calcular las estadísticas, por lo que el programa salta a stats_empty. En este caso se establecen en cero los valores de salida y se finaliza la función.

Si n es diferente de cero, se procede a guardar los registros que deben conservarse. Luego se almacenan en registros de propósito general los valores necesarios para conservarlos durante la ejecución del programa. Una vez guardados los parámetros, se realiza una llamada al sum_array, utilizando nuevamente como entradas la dirección del arreglo y la cantidad de elementos n . El resultado de esta función se devuelve en el registro XMM0, donde se encuentra la suma de todos los elementos del arreglo. Luego con el comando cvtsi2ss se guarda en el registro XMM4 el valor de n en punto flotante, posteriormente se divide la suma almacenada en XMM0 entre dicho valor para obtener la media.

Ahora se limpia el registro eax con una xor para que vuelva a servir de contador. Luego se inicializan los valores mínimo y máximo utilizando el primer elemento del arreglo, XMM2 va a guardar el

valor mínimo del arreglo y XMM3 va a guardar el valor máximo. Se limpia XMM5 que será donde se guarde el acumulador de la varianza. Hasta este punto corresponde el código de la Figura 5.

En la sección stats_loop se recorren uno a uno todos los elementos del arreglo. Primero se compara que $i \geq n$, si lo es hace un salto de línea a stats_done. Seguimos moviendo a xmm1 cada uno de los elementos del array en cada ciclo, con minss y maxss compara es valor XMM1 con el primer valor del array (ambos en XMM2 y XMM3) y se toma respectivamente el valor mínimo y máximo de dicha comparación, este primer dato del array XMM1 se pasa a XMM6, se le resta la media, se multiplica por si mismo y se pasa a XMM5, valor actual en XMM5 es el acumulador de la varianza con solo el primer elementos, se incrementa eax y se repite el ciclo. Cada vez se comparan los nuevos elementos para comprobar si hay un nuevo minimo y máximo y la varianza se ajusta, todo esto hasta que $i \geq n$ y salte a stats_done. En stats_done se tiene que el acumulador de varianzas XMM5 contiene la suma de todas las diferencias cuadráticas. Para obtener la varianza se divide dicho acumulador entre n o XMM4 y se obtiene la verdadera varianza.

Finalmente, la media, varianza, mínimo y máximo se almacenan en las direcciones de memoria proporcionadas por el driver y se devuelven a su lugar el valor de los registros que reservamos. De esta manera, estos cuatro resultados quedan disponibles para las siguientes etapas del procesamiento. Este segunda etapa de código corresponde a la Figura 6.

3.3. normalize_array escalar

```
normalize_array:
; -----
; Guardar mean y stddev en registros SSE adicionales.
;
; xmm8 = mean
; xmm9 = stddev
; -----

movss  xmm8, xmm0          ; xmm8 = mean
movss  xmm9, xmm1          ; xmm9 = stddev
xorps  xmm7, xmm7          ; xmm7 = 0.0
ucomiss xmm0, xmm7
jge    .normalize_copy
xor     eax, eax           ; i = 0

.normalize_loop:
cmp     eax, edx           ; i >= n?
jge     .normalize_done
movss   xmm2, [rdi + rax*4] ; xmm2 = in[i]
subss   xmm2, xmm8         ; xmm2 = in[i] - mean
divss   xmm2, xmm9         ; xmm2 = (in[i] - mean) / stddev
movss   [rsi + rax*4], xmm2 ; out[i] = resultado

inc     eax               ; i++
jmp     .normalize_loop

.normalize_copy:
xor     eax, eax           ; i = 0

.copy_loop:
cmp     eax, edx           ; i >= n?
jge     .normalize_done
movss   xmm2, [rdi + rax*4]
movss   [rsi + rax*4], xmm2

inc     eax
jmp     .copy_loop

.normalize_done:
ret
```

Figura 7: Normalize_array .normalize_loop, .normalize_copy, .copy y .normalize_done

El tercer módulo corresponde a `Normalize_array`, cuyo objetivo es realizar la normalización de los elementos del arreglo utilizando la media y la desviación estándar calculado anteriormente. Esta corresponde al cálculo del Z-score, cuya expresión matemática es:

$$z_i = \frac{x_i - \mu}{\sigma}$$

donde x_i corresponde al elemento actual del arreglo, μ corresponde a la media y σ corresponde a la desviación estándar.

El código de este último módulo se puede apreciar en la Figura 7.

Vamos a refrescar lo que se tiene guardado en los registros tipo XMM, esto se puede apreciar en la tabla del Cuadro 1

Cuadro 1: Resumen valores en los registros

Registro	Dato	Guardado en
XMM0	Media	r13
XMM1	Desviación estandar	–
XMM2	Mínimo	r15
XMM3	Máximo	rbp
XMM4	N en punto flotante	–
XMM5	Varianza	r14

Para poder utilizarlas durante todo el procesamiento se guardan:

Cuadro 2: Valores en los nuevos registros

Registro	Dato	Nuevo Registro
XMM0	Media	XMM8
XMM1	Desviación estandar	XMM9

Ahora la media se guarda en XMM0 y la desviación estandar del array en XMM9, se limpia un registro XMM7 con `xorps` y luego se verifica si la desviación estándar es igual a cero. Esto es necesario debido a que una desviación estándar igual a cero produciría una división entre cero.

Cuando la desviación estándar es diferente de cero, se inicializa el registro utilizado como índice en cero y comienza el ciclo `.normalize_loop`. En cada iteración cuando $i \geq n$, es decir al comparar `”eax”` con `”edx”` y se obtiene un elemento del arreglo mediante su posición:

$$x_i = \text{arr}[i]$$

Este valor se guarda en XMM2, se le resta XMM8 (la media) y se divide por XMM9 (desviación estandar) y se mueve este resultado a la salida. Finalmente, el índice `”eax”` se incrementa en uno y el procedimiento se repite hasta que $i \geq n$

De esta manera, cada valor del arreglo se procesa individualmente y se obtiene un arreglo de salida que contiene los valores normalizados.

En caso de que la desviación estándar sea igual a cero, la función salta a la sección `.normalize_copy`. Este caso ocurre cuando todos los elementos del arreglo tienen el mismo valor, por lo que no es posible realizar directamente la división utilizada en el cálculo del Z-score. En lugar de realizar una división entre cero, el módulo copia directamente los elementos del arreglo de entrada al arreglo de salida. Finalmente, cuando se han procesado todos los elementos, la función llega a `.normalize_done` y retorna al driver.

4. Implementacion vectorial AVX2

La implementacion vectorial se desarrollo utilizando AVX2, ya que permite trabajar con registros YMM de 256 bits. En este proyecto los datos de entrada son numeros en punto flotante, que son de 32 bits. Por lo tanto, un registro YMM puede contener 8 elementos a la vez.

Esto permite que el bucle principal de la version vectorial procese bloques de 8 elementos por iteracion, en lugar de procesar un unico elemento como en la version escalar. Por ejemplo, operaciones como suma, resta, multiplicacion, division, minimo y maximo se aplican a varios carriles del registro mediante instrucciones empaquetadas como `vaddps`, `vsubps`, `vmulps`, `vdivps`, `vminps` y `vmaxps`, para aplicar la misma operacion sobre varios carriles del registro en una sola instruccion.

Tambien se utilizan instrucciones escalares, como `vdivss`, `vaddss` o `vmovss`, cuando el calculo corresponde a un unico valor. Esto ocurre, por ejemplo, al calcular valores finales como la media o la varianza, y tambien en el bucle de remanente, donde se procesan los elementos que no completan un bloque de 8 `float`. Por esta razon, la implementacion combina instrucciones empaquetadas para el camino principal e instrucciones escalares para los casos donde solo hay un dato activo.

Se eligio AVX2 en lugar de SSE porque SSE utiliza registros XMM de 128 bits, capaces de almacenar 4 valores `float`. AVX2 utiliza registros YMM de 256 bits, duplicando el ancho de procesamiento. No se utilizo AVX-512 porque no todas las plataformas de prueba garantizan soporte para registros ZMM de 512 bits.

Por lo tanto, el diseno vectorial se basa en procesar bloques completos de 8 `float` con instrucciones AVX2 y resolver los elementos restantes mediante un bucle escalar de cierre. La Figura 8 muestra la estructura de un registro YMM.

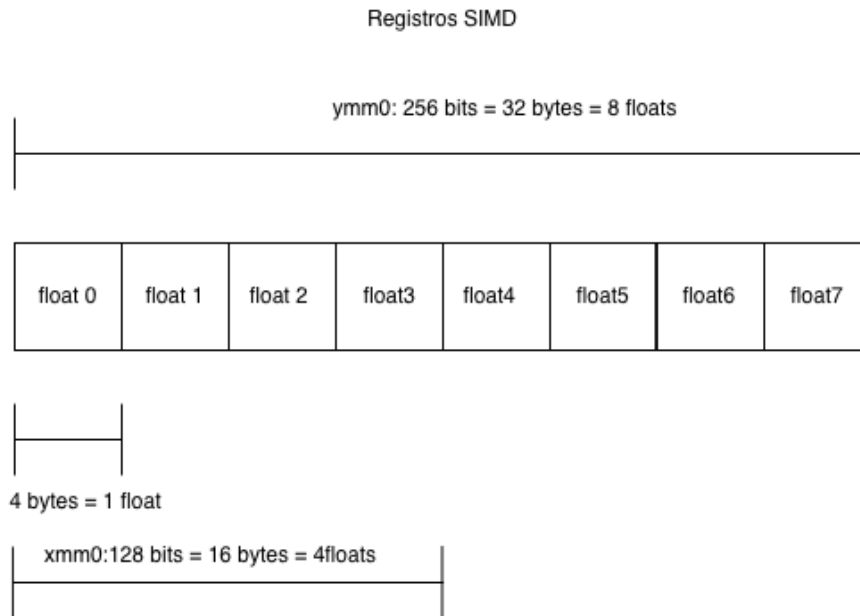


Figura 8: Estructura de un registro YMM de 256 bits, almacena 8 float.

4.1. sum_array vectorial

La funcion `sum_array` vectorial recibe por ABI el puntero al arreglo en `rdi` y el tamaño `n` en `esi`. El resultado se retorna en `xmm0`, siguiendo la convención de retorno para valores `float`. A diferencia de la versión escalar, que acumula un elemento por iteración, la versión AVX2 acumula 8 elementos `float` por iteración utilizando un acumulador vectorial en `ymm0`.

4.1.1. Inicialización y cálculo del límite vectorial

Primero se inicializa el índice `i` en `eax` y se limpia el acumulador vectorial con `vxorps`. Luego se calcula el límite vectorial redondeando `n` hacia abajo al múltiplo de 8 más cercano. Esto se realiza con la operación `and ecx, ~7`. El registro `ecx` contiene entonces la cantidad de elementos que pueden procesarse de forma segura en bloques completos de 8. Luego se evalúa el valor de `ecx`, si este es 0 significa que no hay límite vectorial y se salta a la reducción horizontal.

Listing 1: Inicialización y cálculo del límite vectorial en `sum_array`.

```
sum_array:
    xor     eax, eax                ; eax = i = 0
    vxorps  ymm0, ymm0, ymm0       ; ymm0 = acumulador vectorial = 0

    mov     ecx, esi
    and     ecx, ~7                ; ecx = n redondeado multiplo de 8
    test    ecx, ecx
    jle     .sum_reduce
```

4.1.2. Bucle vectorial de suma

El bucle vectorial se ejecuta mientras `eax < ecx`. En cada iteracion se cargan 8 elementos consecutivos del arreglo mediante `vmovaps`. Esta instruccion se utiliza porque el driver reserva memoria alineada a 32 bytes y el indice vectorial avanza de 8 floats, es decir, 32 bytes por iteracion. Por lo tanto, las direcciones del camino principal mantienen la alineacion requerida. La suma se realiza con `vaddps`, acumulando por carriles dentro de `ymm0`. Cuando `eax >= ecx` se termina el bucle vectorial y se pasa a la reduccion horizontal.

Listing 2: Bucle vectorial de suma.

```
.sum_vec_loop:
    cmp     eax, ecx
    jge     .sum_reduce
    vmovaps ymm1, [rdi + rax*4]    ; carga 8 floats alineados
    vaddps  ymm0, ymm0, ymm1      ; acumula por carril
    add     eax, 8                ; siguiente bloque de 8 floats
    jmp     .sum_vec_loop
```

La Figura 9 resume este proceso: primero se combina la mitad alta y baja del registro YMM, y luego se aplican sumas horizontales dentro del registro XMM hasta obtener el resultado final.

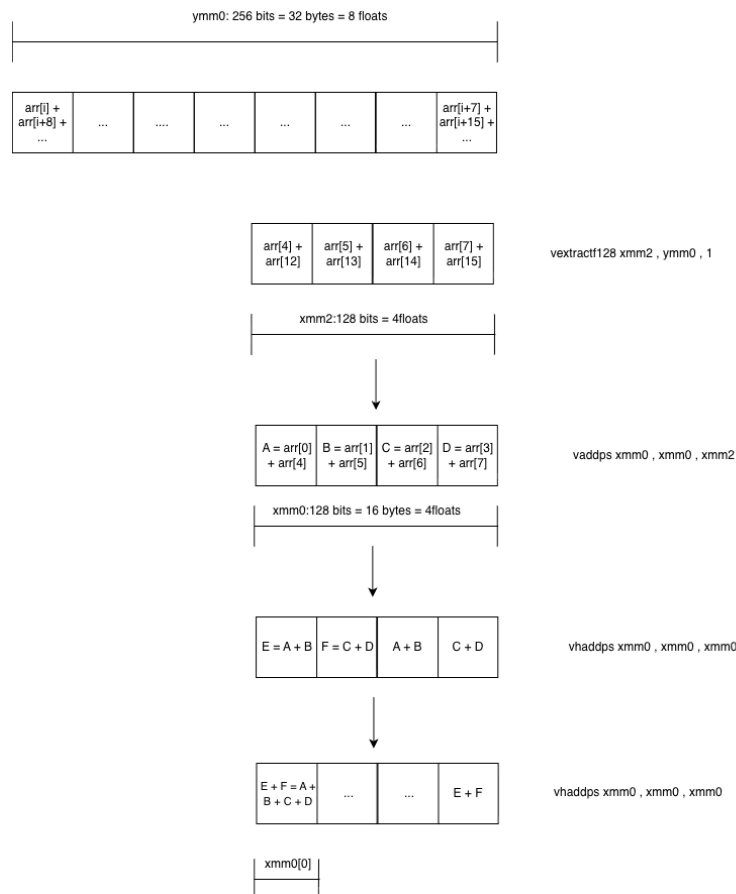


Figura 9: Reduccion horizontal del acumulador vectorial en `sum_array`.

4.1.3. Reduccion horizontal del acumulador vectorial

Despues del bucle vectorial, `ymm0` contiene 8 sumas parciales, una por cada carril vectorial. Por esto se debe realizar una reduccion horizontal. Primero se extrae la mitad alta del registro YMM con `vextractf128` y luego se suma contra la mitad baja con `vaddps`. Finalmente, las dos instrucciones `vhaddps` reducen los 4 carriles restantes hasta dejar la suma total en `xmm0[0]`.

La reduccion horizontal es necesaria porque las instrucciones empaquetadas operan por carril. Al terminar el bucle principal, cada carril contiene una suma parcial de diferentes posiciones del arreglo. La reduccion convierte esas 8 sumas parciales en un unico valor escalar.

Listing 3: Reduccion horizontal del acumulador vectorial.

```
.sum_reduce :
    vextractf128 xmm2, ymm0, 1
    vaddps  xmm0, xmm0, xmm2
    vhaddps xmm0, xmm0, xmm0
    vhaddps xmm0, xmm0, xmm0
```

4.1.4. Bucle escalar para el remanente

Finalmente, si `n` no es multiplo de 8, quedan elementos sin procesar por el bucle vectorial. Estos elementos son el remanente. La funcion los procesa con un bucle escalar usando `vmovss` y `vaddss`. Esto garantiza correctud para cualquier valor de `n`, incluyendo casos como `N=7`, donde no se ejecuta ningun bloque vectorial, o `N=15`, donde se ejecuta un bloque vectorial y luego 7 elementos escalares.

Listing 4: Bucle escalar para el remanente.

```
.sum_scalar_tail :
    cmp     eax, esi
    jge     .sum_done
    vmovss  xmm1, [rdi + rax*4]
    vaddss  xmm0, xmm0, xmm1
    inc     eax
    jmp     .sum_scalar_tail
```

Antes de retornar, se ejecuta `vzeroupper`. Esta instruccion evita penalizaciones al regresar desde codigo AVX, limpiando la parte alta de los registros YMM. El resultado final queda en `xmm0[0]`, como exige la ABI para retornar un `float`.

4.2. compute_stats vectorial

La funcion `compute_stats` calcula cuatro estadisticos del arreglo: media, varianza, minimo y maximo. Por ABI recibe el puntero al arreglo en `rdi`, el tamano `n` en `esi`, y los punteros de salida en `rdx`, `rcx`, `r8` y `r9`. Estos punteros corresponden a `mean`, `var`, `min` y `max`, respectivamente.

La version vectorial mantiene la misma interfaz que la version escalar, pero usa registros YMM para procesar bloques de 8 valores `float`. La funcion realiza dos etapas principales: primero calcula

la media usando `sum_array`, y luego recorre el arreglo para calcular la varianza, el minimo y el maximo. En esta segunda etapa se usa el camino vectorial para los bloques completos y un bucle escalar para el remanente.

4.2.1. Preservacion de argumentos y calculo de la media

Antes de llamar a `sum_array`, la funcion guarda los argumentos importantes en registros callee-saved. Esto es necesario porque la llamada a otra funcion puede modificar registros caller-saved como `rdi`, `rsi`, `rdx`, `rcx`, `r8` y `r9`. Por esta razon, el puntero al arreglo, el tamano y los punteros de salida se preservan en `rbx`, `r12`, `r13`, `r14`, `r15` y `rbp`.

Listing 5: Preservacion de argumentos en `compute_stats`.

```

mov    rbx , rdi          ; rbx = arr
mov    r12d , esi         ; r12d = n
mov    r13 , rdx          ; r13 = mean*
mov    r14 , rcx          ; r14 = var*
mov    r15 , r8           ; r15 = min*
mov    rbp , r9           ; rbp = max*

```

Luego se llama a `sum_array` para obtener la suma total del arreglo. El valor retornado queda en `xmm0`. Para calcular la media, se convierte `n` a punto flotante con `vcvtsi2ss` y se divide la suma entre `float(n)` usando `vdivss`. Esta division es escalar porque la media es un unico valor, no un bloque vectorial.

Listing 6: Calculo de la media.

```

mov    rdi , rbx
mov    esi , r12d
call   sum_array          ; xmm0 = sum

vcvtsi2ss xmm4, xmm4, r12d ; xmm4 = float(n)
vdivss   xmm0, xmm0, xmm4  ; xmm0 = mean
vmovaps  xmm12, xmm0       ; xmm12 = mean escalar
vbroadcastss ymm6, xmm0    ; ymm6 = mean en 8 carriles

```

La instruccion `vbroadcastss` replica la media en todos los carriles de `ymm6`. Esto permite restar la misma media a 8 elementos del arreglo en una sola instruccion vectorial durante el calculo de la varianza.

4.2.2. Inicializacion de acumuladores vectoriales

Despues de calcular la media, se inicializan el indice `i`, el limite vectorial y los acumuladores. Al igual que en `sum_array`, el limite vectorial se obtiene redondeando `n` hacia abajo al multiplo de 8 mas cercano mediante `and ecx, ~7`. El acumulador de varianza se inicializa en cero con `vxorps`.

Tambien se inicializan los acumuladores vectoriales de minimo y maximo. Para esto se replica `arr[0]` en todos los carriles de `ymm10` y `ymm11`. De esta forma, cada carril tiene un valor inicial valido contra el cual comparar los bloques cargados posteriormente.

Listing 7: Inicializacion de acumuladores vectoriales en compute_stats.

```

xor      eax, eax                ; eax = i = 0
mov      ecx, r12d
and      ecx, ~7                  ; ecx = limite vectorial
vxorps   ymm7, ymm7, ymm7        ; ymm7 = acc_var vectorial = 0
vbroadcastss ymm10, [rbx]        ; ymm10 = min inicial = arr[0]
vbroadcastss ymm11, [rbx]        ; ymm11 = max inicial = arr[0]

test     ecx, ecx
jle      .stats_no_vec_blocks

```

Si el limite vectorial es cero, significa que $n < 8$. En ese caso no existe ningun bloque completo de 8 elementos y la funcion salta directamente al camino escalar. Esto ocurre, por ejemplo, con $N=7$.

4.2.3. Bucle vectorial de varianza, minimo y maximo

El bucle vectorial se ejecuta mientras $eax < ecx$. En cada iteracion se cargan 8 valores del arreglo usando `vmovaps`. Esta carga es alineada porque el driver reserva memoria con alineacion de 32 bytes y el indice avanza de 8 en 8 elementos, es decir, 32 bytes por iteracion.

Dentro del bucle se actualizan tres resultados parciales. Primero, `vminps` y `vmaxps` comparan por carril el bloque actual contra los acumuladores vectoriales de minimo y maximo. Despues, para la varianza, se calcula la diferencia entre cada elemento y la media con `vsubps`; luego se eleva al cuadrado con `vmulps`; finalmente se acumula en `ymm7` usando `vaddps`.

Listing 8: Bucle vectorial de compute_stats.

```

.stats_vec_loop:
cmp      eax, ecx
jge      .stats_reduce_vec
vmovaps   ymm8, [rbx + rax*4]      ; ymm8 = arr[i..i+7]
vminps   ymm10, ymm10, ymm8        ; min por carril
vmaxps   ymm11, ymm11, ymm8        ; max por carril
vsubps   ymm9, ymm8, ymm6          ; diff = x - mean
vmulps   ymm9, ymm9, ymm9          ; diff^2
vaddps   ymm7, ymm7, ymm9          ; acc_var += diff^2 por carril
add      eax, 8
jmp      .stats_vec_loop

```

Es importante notar que los acumuladores `ymm10` y `ymm11` no contienen todavia el minimo y maximo globales. Contienen minimos y maximos parciales por carril. Por ejemplo, el carril 0 acumula comparaciones contra los elementos `arr[0]`, `arr[8]`, `arr[16]`, etc. Por esta razon, al finalizar el bucle vectorial se requiere una reduccion horizontal para obtener un unico minimo, un unico maximo y una unica suma de cuadrados.

4.2.4. Reduccion horizontal de la varianza

La varianza se acumula en `ymm7` como ocho sumas parciales. Para convertirlas en un unico escalar se aplica una reduccion horizontal similar a la utilizada en `sum_array`. Primero se extrae la mitad alta del registro YMM, luego se suma contra la mitad baja y finalmente se aplican dos instrucciones `vhaddps`. El resultado queda en `xmm5[0]`.

Listing 9: Reduccion horizontal de la varianza parcial.

```
.stats_reduce_vec:
    vextractf128 xmm5, ymm7, 1
    vaddps  xmm5, xmm5, xmm7
    vhaddps xmm5, xmm5, xmm5
    vhaddps xmm5, xmm5, xmm5
```

En este punto, `xmm5[0]` contiene la suma total de los terminos $(x - \text{mean})^2$ procesados por el bucle vectorial. La division final entre `n` se realiza al final de la funcion mediante `vdivss`, porque la varianza final es un unico valor escalar.

4.2.5. Reduccion horizontal de minimo y maximo

La reduccion de minimo y maximo usa una idea similar, pero en lugar de sumar carriles se comparan entre si. Primero se extrae la mitad alta con `vextractf128`. Luego `vminps` o `vmaxps` combinan la mitad alta y baja. Despues se usan permutaciones con `vpermilps` para reordenar los carriles y seguir comparando hasta que el minimo o maximo global quede en el carril bajo del registro.

Listing 10: Reduccion horizontal del minimo y maximo.

```
; Reducir min: 8 carriles → xmm2[0]
vextractf128 xmm2, ymm10, 1
vminps  xmm2, xmm2, xmm10
vpermilps xmm1, xmm2, 0b10110001
vminps  xmm2, xmm2, xmm1
vpermilps xmm1, xmm2, 0b01001110
vminps  xmm2, xmm2, xmm1

; Reducir max: 8 carriles → xmm3[0]
vextractf128 xmm3, ymm11, 1
vmaxps  xmm3, xmm3, xmm11
vpermilps xmm1, xmm3, 0b10110001
vmaxps  xmm3, xmm3, xmm1
vpermilps xmm1, xmm3, 0b01001110
vmaxps  xmm3, xmm3, xmm1
```

Al finalizar esta etapa, `xmm2[0]` contiene el minimo parcial reducido y `xmm3[0]` contiene el maximo parcial reducido. Si el arreglo tiene remanente, estos valores todavia pueden ser actualizados por el bucle escalar final.

4.2.6. Bucle escalar para el remanente y almacenamiento de resultados

Despues de la reduccion vectorial, la funcion entra al bucle escalar para procesar los elementos restantes. Esto ocurre cuando n no es multiplo de 8. El indice `eax` ya contiene el primer elemento no procesado por el bucle vectorial. Para cada elemento del remanente se actualizan minimo, maximo y acumulador de varianza usando instrucciones escalares.

Listing 11: Bucle escalar de remanente en `compute_stats`.

```
.stats_scalar_tail:
    cmp     eax, r12d
    jge     .stats_done
    vmovss  xmm1, [rbx + rax*4]      ; xmm1 = x
    vminss  xmm2, xmm2, xmm1        ; min = min(min, x)
    vmaxss  xmm3, xmm3, xmm1        ; max = max(max, x)
    vsubss  xmm6, xmm1, xmm12       ; diff = x - mean
    vmulss  xmm6, xmm6, xmm6        ; diff^2
    vaddss  xmm5, xmm5, xmm6        ; acc_var += diff^2
    inc     eax
    jmp     .stats_scalar_tail
```

Finalmente, la varianza se obtiene dividiendo el acumulador total entre `float(n)`. Este paso se realiza con `vdivss`, ya que el resultado final es escalar. Luego se almacenan los cuatro resultados en las direcciones recibidas por ABI.

Listing 12: Calculo final y almacenamiento de estadisticos.

```
.stats_done:
    vdivss  xmm5, xmm5, xmm4        ; var = acc_var / float(n)
    vmovss  [r13], xmm12            ; *mean = mean
    vmovss  [r14], xmm5            ; *var = var
    vmovss  [r15], xmm2            ; *min = min
    vmovss  [rbp], xmm3            ; *max = max
```

De esta forma, `compute_stats` combina procesamiento vectorial para los bloques completos con procesamiento escalar para el remanente. El camino vectorial acelera el calculo de varianza, minimo y maximo sobre bloques de 8 elementos, mientras que el tail escalar garantiza que todos los elementos sean considerados, incluso cuando n no es multiplo de 8.

4.3. normalize_array vectorial

La funcion `normalize_array` recibe por ABI el puntero al arreglo de entrada en `rdi`, el puntero al arreglo de salida en `rsi`, el tamano n en `edx`, la media en `xmm0` y la desviacion estandar en `xmm1`. Su objetivo es escribir en el arreglo de salida el valor normalizado de cada elemento:

$$out[i] = \frac{in[i] - mean}{stddev}$$

La version vectorial procesa bloques de 8 valores float usando registros YMM. Para esto replica la media y la desviacion estandar en todos los carriles de registros vectoriales, de forma que cada bloque de 8 elementos pueda normalizarse con instrucciones empaquetadas. Ademas, la funcion considera el caso borde `stddev == 0.0`, en este caso no se realiza la division y se copia el arreglo de entrada hacia el arreglo de salida para evitar division por cero.

4.3.1. Inicializacion y deteccion del caso `stddev == 0`

La funcion inicia colocando el indice `i` en cero y calculando el limite vectorial. Que permite que el bucle vectorial procese solo bloques completos de 8 elementos.

Antes de normalizar, se compara la desviacion estandar contra cero. Para esto se limpia `xmm2` con `vxorps` y se usa `vucomiss` para comparar `xmm1` contra cero. Si `stddev == 0.0`, la funcion salta al camino de copia, evitando una division por cero.

Listing 13: Inicializacion y deteccion de `stddev == 0`.

```
normalize_array:
    xor     eax, eax                ; eax = i = 0
    mov     ecx, edx
    and     ecx, ~7                ; ecx = limite vectorial

    vxorps  xmm2, xmm2, xmm2        ; xmm2 = 0.0
    vucomiss xmm1, xmm2             ; stddev == 0.0?
    je      .norm_copy_vec_loop
```

4.3.2. Preparacion del camino vectorial

Si la desviacion estandar no es cero, la funcion prepara los registros necesarios para el camino de normalizacion. La media y la desviacion estandar se conservan en `xmm8` y `xmm9` para ser usadas posteriormente en el remanente escalar. Luego, ambas se replican en registros YMM mediante `vbroadcastss`. Esto permite restar la misma media y dividir entre la misma desviacion estandar para 8 elementos simultaneamente.

Listing 14: Broadcast de media y desviacion estandar.

```
vmovaps xmm8, xmm0                ; xmm8[0] = mean
vmovaps xmm9, xmm1                ; xmm9[0] = stddev
vbroadcastss ymm10, xmm0           ; ymm10 = [mean, ..., mean]
vbroadcastss ymm11, xmm1           ; ymm11 = [stddev, ..., stddev]
```

4.3.3. Bucle vectorial de normalizacion

El bucle vectorial se ejecuta mientras `eax < ecx`, o sea mientras `i` sea mayor que el limite vectorial. En cada iteracion se cargan 8 elementos consecutivos desde el arreglo de entrada usando `vmovaps`. Luego se aplica la formula de normalizacion usando instrucciones empaquetadas: `vsubps` resta la

media replicada y `vdivps` divide entre la desviacion estandar replicada. Finalmente, el resultado se almacena en el arreglo de salida con `vmovaps`.

Listing 15: Bucle vectorial de normalizacion.

```
.norm_vec_loop:
    cmp     eax, ecx
    jge     .norm_scalar_tail
    vmovaps ymm12, [rdi + rax*4]    ; ymm12 = in[i..i+7]
    vsubps  ymm12, ymm12, ymm10    ; ymm12 = in - mean
    vdivps  ymm12, ymm12, ymm11    ; ymm12 = (in - mean) / stddev
    vmovaps [rsi + rax*4], ymm12   ; out[i..i+7] = resultado
    add     eax, 8
    jmp     .norm_vec_loop
```

El uso de `vmovaps` es valido porque tanto el arreglo de entrada como el arreglo de salida se reservan alineados a 32 bytes en el driver C. Ademas, el indice del bucle vectorial avanza de 8 floats por iteracion, es decir, 32 bytes. Por esta razon, las direcciones usadas en el camino principal permanecen alineadas.

4.3.4. Bucle escalar para el remanente

Si `n` no es multiplo de 8, despues del bucle vectorial quedan elementos sin procesar. Estos elementos se normalizan con un bucle escalar usando instrucciones `vmovss`, `vsubss` y `vdivss`. En este caso solo se opera sobre un valor float por iteracion, por lo que se usan registros XMM y operaciones escalares.

Listing 16: Bucle escalar de remanente en `normalize_array`.

```
.norm_scalar_tail:
    cmp     eax, edx
    jge     .norm_done
    vmovss  xmm12, [rdi + rax*4]    ; xmm12 = in[i]
    vsubss  xmm12, xmm12, xmm8      ; xmm12 = in[i] - mean
    vdivss  xmm12, xmm12, xmm9      ; xmm12 = (in[i] - mean) / stddev
    vmovss  [rsi + rax*4], xmm12   ; out[i] = xmm12
    inc     eax
    jmp     .norm_scalar_tail
```

4.3.5. Camino de copia cuando `stddev == 0`

Cuando la desviacion estandar es cero, todos los elementos del arreglo tienen el mismo valor. En ese caso la normalizacion $(x - mean)/stddev$ implicaria una division por cero. Para evitar este caso, se copia el arreglo de entrada hacia el arreglo de salida. Este camino tambien tiene una version vectorial para bloques completos de 8 elementos y una version escalar para el remanente.

Listing 17: Copia vectorial cuando `stddev == 0`.

```

.norm_copy_vec_loop:
    cmp    eax, ecx
    jge    .norm_copy_scalar_tail
    vmovaps ymm12, [rdi + rax*4]    ; ymm12 = in[i..i+7]
    vmovaps [rsi + rax*4], ymm12    ; out[i..i+7] = in[i..i+7]
    add    eax, 8
    jmp    .norm_copy_vec_loop

```

Para los elementos restantes, se utiliza una copia escalar con vmovss:

Listing 18: Copia escalar del remanente cuando stddev == 0.

```

.norm_copy_scalar_tail:
    cmp    eax, edx
    jge    .norm_done
    vmovss  xmm12, [rdi + rax*4]    ; xmm12 = in[i]
    vmovss  [rsi + rax*4], xmm12    ; out[i] = in[i]
    inc    eax
    jmp    .norm_copy_scalar_tail

```

5. Casos de prueba y verificacion de correctud

Para verificar la correctud de la implementacion se utilizaron casos de prueba para cubrir el funcionamiento normal y tambien los principales casos borde del programa. Estos casos incluyen arreglos vacios, arreglos de un solo elemento, tamanos no multiplos de 8, bloques vectoriales exactos, arreglos constantes y arreglos con valores negativos.

La verificacion se baso en el script `verify_reference.py`, se adaptó para comparar tambien el arreglo normalizado. Ademas, se agrego el comando `make check` al Makefile para ejecutar automaticamente la matriz de casos. Este comando genera los archivos de entrada, ejecuta la version escalar y la version vectorial, y compara los resultados contra una referencia calculada en Python.

La comparacion incluye los estadisticos producidos por `compute_stats` y el arreglo normalizado generado por `normalize_array`. La Tabla 3 resume valores representativos de la verificacion. En particular, se muestran la media μ y la desviacion estandar σ , ya que estos valores dependen de la suma, la varianza y el manejo correcto de casos como `stddev = 0`.

Cuadro 3: Validacion de correctud para casos de prueba

Caso	Referencia		Escalar		Vectorial		Resultado
	μ	σ	μ	σ	μ	σ	
empty	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	PASA
n1	59.330193	0.000000	59.330193	0.000000	59.330193	0.000000	PASA
n7	40.380961	42.780877	40.380959	42.780876	40.380959	42.780876	PASA
n8	24.767264	57.997111	24.767265	57.997116	24.767263	57.997109	PASA
n15	-0.060861	56.733859	-0.060861	56.733860	-0.060861	56.733860	PASA
n16	1.475429	32.356933	1.475429	32.356934	1.475429	32.356934	PASA
n1000	-0.442242	57.350445	-0.442242	57.350449	-0.442242	57.350449	PASA
constant16	5.000000	0.000000	5.000000	0.000000	5.000000	0.000000	PASA
edge15	-66666.666667	573488.351136	-66666.664100	573488.312000	-66666.664100	573488.312000	PASA
edge1001	0.000000	534522.483825	0.000000	534523.125000	0.000000	534522.438000	PASA

Como se observa en la Tabla 3, ambas versiones pasan todos los casos evaluados. Las diferencias numericas observadas son pequenas en relacion con la tolerancia definida y se deben al redondeo de punto flotante y al distinto orden de acumulacion entre la version escalar y la version vectorial.

Aunque la tabla muestra solo la media y la desviacion estandar como valores representativos, la herramienta de verificacion tambien compara el resto de los estadisticos y el arreglo normalizado completo. Por tanto, la prueba valida el flujo completo del programa, incluyendo `sum_array`, `compute_stats` y `normalize_array`.

6. Resultados de Rendimiento

Para evaluar el rendimiento de las versiones escalar y vectorial se midi6 el tiempo promedio de ejecuci6n del kernel para distintos tama6os de entrada. Cada medici6n se realiz6 con 30 repeticiones, reportando tanto el promedio como la desviaci6n estandar. Adem6s, se calcul6 el *speedup* como la raz6n entre el tiempo promedio de la versi6n escalar y el tiempo promedio de la versi6n vectorial:

$$\text{speedup} = \frac{T_{\text{escalar}}}{T_{\text{vectorial}}}$$

La Tabla 4 resume los resultados obtenidos. A partir de estos resultados se observa que la versi6n vectorial es consistentemente m6s r6pida que la versi6n escalar para todos los tama6os evaluados. Esto confirma que el uso de instrucciones AVX2 permite procesar varios elementos del arreglo por instrucci6n, reduciendo la cantidad total de iteraciones necesarias en las secciones vectorizables del programa.

Cuadro 4: Resultados de rendimiento para la versión escalar y vectorial

N	T_{esc} (ms)	σ_{esc} (ms)	T_{vec} (ms)	σ_{vec} (ms)	Speedup
10^3	0.003337	0.000011	0.000326	0.000013	10.24x
10^5	0.338722	0.014186	0.037825	0.007130	8.95x
10^6	3.447359	0.212859	0.400483	0.100981	8.61x
5×10^7	167.231311	2.180705	38.287832	1.568045	4.37x

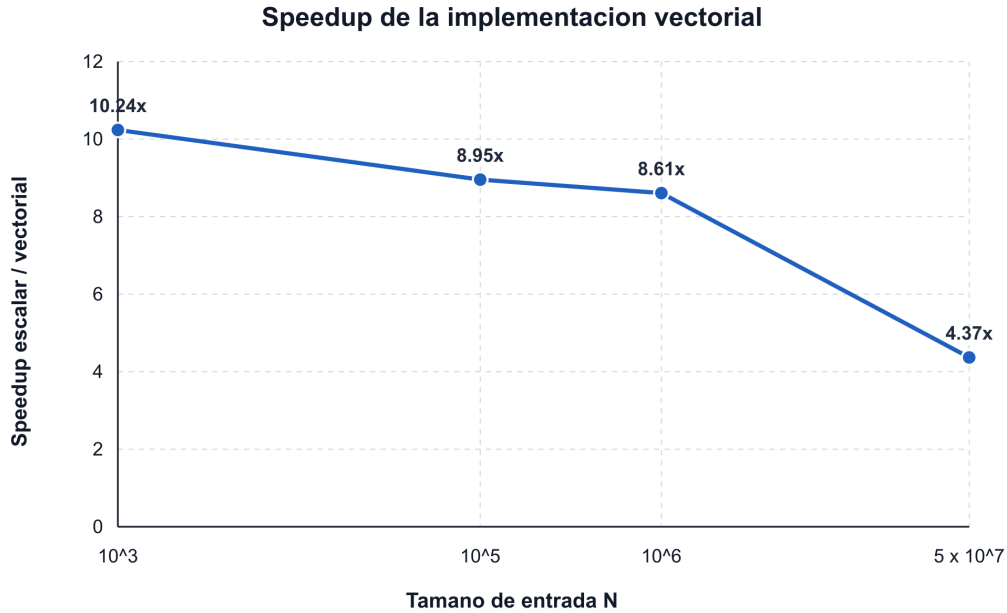


Figura 10: Speedup de la versión vectorial respecto a la versión escalar para distintos tamaños de entrada.

Para $N = 10^3$ se obtiene el mayor speedup medido, aproximadamente $10,24\times$. En este caso, el tamaño de entrada ya es suficiente para aprovechar el paralelismo SIMD, pero todavía no domina completamente el costo de mover grandes cantidades de datos en memoria. Para $N = 10^5$ y $N = 10^6$, el speedup se mantiene alto, alrededor de $8,95\times$ y $8,61\times$, respectivamente. Esto indica que la implementación vectorial escala de forma adecuada conforme aumenta el número de elementos.

En el caso de $N = 5 \times 10^7$, el speedup disminuye a aproximadamente $4,37\times$. Esta reducción no implica que la versión vectorial deje de ser efectiva; más bien sugiere que, para entradas muy grandes, el rendimiento empieza a estar más limitado por el acceso a memoria y por el movimiento de datos que por la cantidad de operaciones aritméticas. En otras palabras, aunque AVX2 reduce el número de instrucciones aritméticas necesarias, el programa sigue teniendo que leer y escribir una cantidad grande de datos.

También se observa que la desviación estándar es pequeña en relación con los tiempos promedio, especialmente para los tamaños grandes. Esto indica que las mediciones son suficientemente

estables para comparar ambas versiones.

7. Evidencia de la sesión de GDB

7.1. Evidencia de GDB Escalar

Esta sección consiste en la demostración del funcionamiento paso a paso de la sección escalar del proceso. Comprobar el estado de los registros según lo explicado en su sección correspondiente. Para inicializar este segmento también llamado Debugger, En el panel de control WSL se abre la carpeta donde están los archivos y el ejecutable, de manera que se vea como la Figura , antes de correr el GDB se confirma la carga correcta de la siguiente manera:

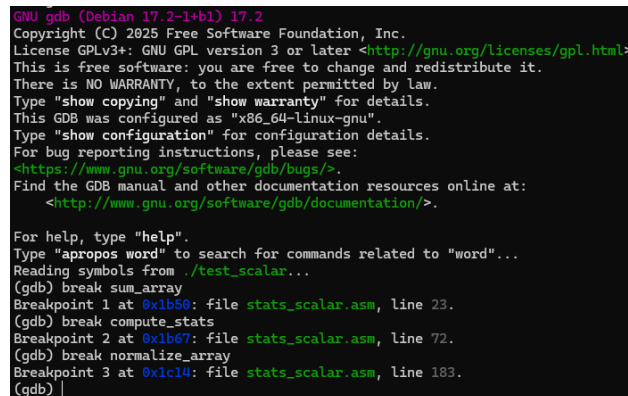
```
nasm -felf64 -g -F dwarf stats_scalar.asm -o stats_scalar.o
```

```
gcc -g -O0 driver.c stats_scalar.o -lm -o test_scalar
```

De esta manera se crean los archivos correspondientes para poder vincular. Ahora se procede a inicializar el GDB. En primera instancia se ejecuta:

```
gdb./test_scalar
```

Esto inicializa el modo Debugger, se ha de ver como la Figura 11



```
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./test_scalar...
(gdb) break sum_array
Breakpoint 1 at 0x1b50: file stats_scalar.asm, line 23.
(gdb) break compute_stats
Breakpoint 2 at 0x1b67: file stats_scalar.asm, line 72.
(gdb) break normalize_array
Breakpoint 3 at 0x1c14: file stats_scalar.asm, line 183.
(gdb) |
```

Figura 11: Inicialización del GDB Escalar

Una vez dentro del modo Debugger lo siguiente es definir los break points. Los break points son puntos donde el programa se detiene dando la oportunidad de correr el resto del código desde ahí o avanzar línea por línea en el archivo .asm.

Los break points se definen de la forma `break[nombre_modulo]`. Para el caso de la prueba se van a definir 3 ubicaciones de parada distintas para comprobar el funcionamiento: En `sum_array`, en `compute_stats` y en `normalize_array`, estos también se pueden apreciar ya seleccionados en la Figura 11

7.1.1. GDB en sum_array Escalar

Este segmento define la sección de código dada por

Listing 19: Código sum_array escalar

```
sum_array:

    xor     eax, eax                ; i = 0
    xorps   xmm0, xmm0             ; xmm0 = 0.0 (Acumulador)

.sum_loop:

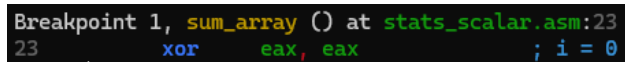
    cmp     eax, esi               ; i >= n?
    jge     .sum_done
    movss   xmm1, [rdi + rax*4]     ; xmm1 = arr[i]
    addss   xmm0, xmm1             ; suma += arr[i]
    inc     eax                    ; i++
    jmp     .sum_loop

.sum_done:
    ret
```

O visto de manera más completa en la Figura 4

Ya que su funcionalidad se definió en la sección del sum_array escalar, solamente se va a comprobar

Se ejecuta el comando *runinput.datoutput.dat100* Esto porque se requiere también un archivo de entrada y uno de salida que también ya fueron cargados en la carpeta actual. Se detiene entonces en la primera línea del GDB como se aprecia en la Figura 12



```
Breakpoint 1, sum_array () at stats_scalar.asm:23
23      xor     eax, eax                ; i = 0
```

Figura 12: Primera línea del GDB sum_array escalar

El break point se detuvo en el punto correcto, ahora se corre hacia las siguientes líneas usando el comando correspondiente. Tras completar un ciclo de líneas del GDB se aprecia el siguiente comportamiento de registros en la Figura ??

Figura 13: Primer ciclo del GDB sum_array escalar

Tras varias interacción se obtiene el código de la Figura 14

Figura 14: Ciclo completado del GDB sum_array escalar

25

7.1.2. GDB en compute_stats Escalar

Este segmento define la sección de código dada por

Listing 20: Código compute_stats escalar

compute_stats :

```
test    esi , esi
jz      .stats_empty

push    rbx
push    rbp
push    r12
push    r13
push    r14
push    r15
sub     rsp , 8

mov     rbx , rdi                ; rbx = arr
.
.
.
.
```

ret

O visto de manera más completa en las Figuras 5 y 6

Ya que su funcionalidad se definió en la sección del compute_stats escalar, solamente se va a comprobar su funcionamiento

El código ya está activo para este punto, pero si se vuelve a correr con el mismo comando *runinput.datoutput.dat100*, entonces va a saltar hasta el siguiente break point como se aprecia en la Figura 15



```
Breakpoint 2, compute_stats () at stats_scalar.asm:72
72      test    esi, esi
(gdb) |
```

Figura 15: Primera línea del GDB compute_stats escalar

Primero se hace el acomodo de los valores de los registros a usar según ya se había descrito anteriormente y se llama al sum_array como se aprecia en la Figura 16 luego de comprobar el contenido de los registros

```

Breakpoint 2, compute_stats () at stats_scalar.asm:72
72      test     esi, esi
(gdb)
73      jz       .stats_empty
(gdb)
75      push    rbx
(gdb)
compute_stats () at stats_scalar.asm:76
76      push    rbp
(gdb)
compute_stats () at stats_scalar.asm:77
77      push    r12
(gdb)
compute_stats () at stats_scalar.asm:78
78      push    r13
(gdb)
compute_stats () at stats_scalar.asm:79
79      push    r14
(gdb)
compute_stats () at stats_scalar.asm:80
80      push    r15
(gdb)
compute_stats () at stats_scalar.asm:83
83      sub     rsp, 8
(gdb)
compute_stats () at stats_scalar.asm:85
85      mov     rbx, rdi          ; rbx = arr
(gdb)
86      mov     r12d, esi         ; r12d = n
(gdb)
87      mov     r13, rdx          ; r13 = mean*
(gdb)
88      mov     r14, rcx          ; r14 = var*
(gdb)
89      mov     r15, r8           ; r15 = min*
(gdb)
90      mov     rbp, r9           ; rbp = max*
(gdb)
92      mov     rdi, rbx          ; argumento arr
(gdb)
93      mov     esi, r12d         ; argumento n
(gdb)
94      call    sum_array         ; xmm0 = sum(arr)
(gdb) info registers rbx r12d r13 r14 r15 rbp rdi esi
rbx      0x55555555a200          93824992256512
r12d     0x10                  16
r13      0x7fffffffdbb0        140737488346032
r14      0x7fffffffdbac        140737488346028
r15      0x7fffffffdba8        140737488346024
rbp      0x7fffffffdba4        0x7fffffffdba4
rdi      0x55555555a200          93824992256512
esi      0x10                  16
(gdb) |

```

Figura 16: Declaración de registros del GDB compute_stats escalar

Hace llama al sum_array para tener la suma de XMM0 y lo devuelve. El comportamiento es el esperado en el proceso de la suma. Tras devolver XMM0 se comprueban los pasos siguientes de la Figura 17 antes de entrar al stats loop.

```

sum_array.sum_done () at stats_scalar.asm:106
ret
(gdb) next
compute_stats () at stats_scalar.asm:190
; cvtsi128 xmm0, r12d
; xmm0 = (float)n
(gdb) next
divss xmm0, xmm0          ; xmm0 = mean
(gdb) info registers xmm0
xmm0 {v8_float16 = {0x8000, 0x4009, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v8_half = {0x8000, 0x4009, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v4_float = {0x4009, 0x0, 0x0, 0x0}, v2_double = {0x40090000, 0x0}, v4_int8 = {0x0, 0x20, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v4_int16 = {0x8000, 0x4009, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v4_int32 = {0x40090000, 0x0, 0x0, 0x0}, v2_int64 = {0x40090000, 0x0}, uint128 = {0x40090000, 0x0}, uint64 = {0x40090000, 0x0}, uint32 = {0x40090000, 0x0}, uint16 = {0x40090000, 0x0}, uint8 = {0x40090000, 0x0}, int128 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, int64 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, int32 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, int16 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, int8 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float16 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float32 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float64 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, bfloat16 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, bfloat32 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, bfloat64 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float128 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float256 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float512 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float1024 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float2048 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float4096 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float8192 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float16384 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float32768 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float67536 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float135072 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float270144 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float540288 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float1080576 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float2161152 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float4322304 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float8644608 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float17289216 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float34578432 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float69156864 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float138313728 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float276627456 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float553254912 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float1106509824 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float2213019648 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float4426039296 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float8852078592 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float17704157184 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float35408314368 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float70816628736 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float141633257472 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float283266514944 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float566533029888 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float1133066059776 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float2266132119552 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float4532264239104 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float9064528478208 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float18129056956416 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float36258113912832 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float72516227825664 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float145032455651328 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float290064911302656 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float580129822605312 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float1160259645210624 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float2320519290421248 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float4641038580842496 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float9282077161684992 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float18564154323369984 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float37128308646739968 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float74256617293479936 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float148513234586959872 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float297026469173919744 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float594052938347839488 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float1188105877695678976 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float2376211755391357952 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float4752423510782715904 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float9504847021565423808 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float19009694043130847616 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float38019388086261695232 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float76038776172523390464 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float152077552345046780928 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float304155104690133561856 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float608310209380267123712 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float1216620418760534454272 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float2433240837521068908544 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float4866481675042137817088 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float9732963350084275634176 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float19465926700165511268352 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float38931853400331022536704 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float77863706800662045073408 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float155727413601324090146816 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float311454827202648180293632 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float622909654410176360587264 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float12458193088033277211744 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float24916386176066554423488 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float49832772352133108846976 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float99665544704266217693952 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float19933108940853243537904 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float39866217881610487075808 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float79732435763220974151616 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float159464871524441923033312 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float318929743048883846066624 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float637859486097767692133248 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float1275718972195535384266496 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float2551437944391070768532992 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float5102875888782141537065984 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float10205751775642283074131968 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float20411503551284566148263936 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float40823007102569132296527872 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float81646014205138264593055648 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float163292028410256491181111296 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float326584056820513272222225792 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float653168113641025544444451584 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float1306336227210511088888903168 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float2612672454421022177777806336 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float5225344908842424355555612672 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float1045068981648484871111132544 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float2090137963296969742222265088 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float4180275926593939484444530176 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float8360551853187878968889060352 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float16721103706757557937778120704 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float334422074135151158755562414144 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float668844148270302317511124828288 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float133768829654060463502225656576 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float267537659308120927004451331152 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float535075318616217844008902662304 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float107015063732423568801795332448 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float214030127464847137603590664896 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float428060254929694275207181329792 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float856120509859388550414262659584 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float1712241019718777100828553191168 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float3424482039437554201657106382336 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float6848964078875108403211412764672 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float13697928157750216806422825529344 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float27395856315500433612845651055888 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float54791712631000867225691311117776 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float109583425262001734451218262235552 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float219166850524003469024365345111104 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float438333701048006938048730690222208 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float876667402096013876097461380444416 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float175333480419202755219492276888832 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float350666960838405510438984553777664 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float701333921676811020877979107555328 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float1402667843536222041559958151110656 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float2805335687072444083119917022221312 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float5610671374144888166239834044442624 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float1122134274828977732479668088889248 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float2244268549657955464959336177778496 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float4488537099315910929918672355556992 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float8977074198631821859837344711113984 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float179541483972636371197666888422227968 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float35908296794527274239533377684444593728 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float71816593589054548479066755368889187456 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float143633187179109096958132717778375008 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float287266374358218193916265435556750016 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float57453274871643638783253087111350032 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float114906549743287757566506174222260064 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float2298130994865755151330122344444120128 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float459626198973151030266044688888240256 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float919252397946302060532089377777480512 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float183850479589260412106417787555596102016 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float36770095917852082421283557511119220416 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float735401918357041648425671150222444404416 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float1470803836714083296851322450444888808832 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float2941607673428166593702644900889777617664 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float5883215346856333187405289801779555333328 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float11766430693712666375810579603559111110656 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float23532861387425332751621159207118222221312 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float47065722774850665503242304424444442624 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float94131445549701331006484608848888885248 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float188262891094026662012897216977777704896 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float376525782188053324025794433955555407936 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float7530515643761066480515888679111111818784 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float15061031287522131680103773578222223637568 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float30122062575044263360207547156444447275136 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float602441251500885267204151043128888845502784 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float1204882503001770534408222086257777711005568 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float2409765006003541068816441714555554220111111 = {0x7ffff, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, float48195300120070821376328834291111
```

```

compute_stats.stats_loop () at stats_scalar.asm:111
(gdb) info registers xmm0 xmm1 xmm2
xmm0 {v0.float16 = {0x10, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v0_half = {0x10, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v0_float = {0x10, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v2_double = {0x10, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v2_int32 = {0x10, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v2_int64 = {0x10, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}}
xmm1 {v0.float16 = {0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v0_half = {0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v0_float = {0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v2_double = {0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v2_int32 = {0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v2_int64 = {0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}}
xmm2 {v0.float16 = {0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v0_half = {0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v0_float = {0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v2_double = {0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v2_int32 = {0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v2_int64 = {0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}}

```

Figura 18: Inicio del stats_loop del GDB compute_stats escalar

Hasta completar el ciclo de los estadísticos. El cálculo paso a paso de los estadísticos y la confirmación de su posición en los registros esperados se puede apreciar en la Figura 19

```

(gdb) movss xmm1, [rbx + rax*4]; xmm1 = arr[i]
(gdb) minss xmm1, xmm1; min = min(min, arr[i])
(gdb) maxss xmm1, xmm1; max = max(max, arr[i])
(gdb) movss xmm0, xmm1; xmm0 = arr[i] - mean
(gdb) subss xmm0, xmm0; xmm0 = (arr[i] - mean)*2
(gdb) mulss xmm0, xmm0; xmm0 = (arr[i] - mean)^2
(gdb) addss xmm0, xmm0; acc_var += ...
(gdb) inc esi; i++
(gdb) jmp stats_loop
(gdb) info registers eax esi
eax {v0.float16 = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v0_half = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v0_float = {0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v2_double = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v2_int32 = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v2_int64 = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}}
esi {v0.float16 = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v0_half = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v0_float = {0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v2_double = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v2_int32 = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v2_int64 = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}}
(gdb) nexti
(gdb) cap eax, r14d; i += n?
(gdb) jge stats_done
(gdb) compute_stats.stats_done () at stats_scalar.asm:111
(gdb) divss xmm0, xmm0; var = acc_var / n
(gdb) movss [r13], xmm0; *mean = mean
(gdb) movss [r14], xmm0; *var = var
(gdb) movss [r15], xmm0; *min = min
(gdb) movss [rbp], xmm0; *max = max
(gdb) add rbp, 4
(gdb) info registers xmm0 xmm1 xmm2
xmm0 {v0.float16 = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v0_half = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v0_float = {0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v2_double = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v2_int32 = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v2_int64 = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}}
xmm1 {v0.float16 = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v0_half = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v0_float = {0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v2_double = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v2_int32 = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v2_int64 = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}}
xmm2 {v0.float16 = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v0_half = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v0_float = {0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v2_double = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v2_int32 = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v2_int64 = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}}
(gdb) nexti
(gdb) info registers r13 r14 r15 rbp
r13 {v0.float16 = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v0_half = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v0_float = {0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v2_double = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v2_int32 = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v2_int64 = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}}
r14 {v0.float16 = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v0_half = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v0_float = {0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v2_double = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v2_int32 = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v2_int64 = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}}
r15 {v0.float16 = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v0_half = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v0_float = {0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v2_double = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v2_int32 = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v2_int64 = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}}
rbp {v0.float16 = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v0_half = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v0_float = {0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v2_double = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v2_int32 = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v2_int64 = {0x0, 0x120, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}}
(gdb) |

```

Figura 19: Final del stats_loop del GDB compute_stats escalar

Por ultimo se guardan los estadísticos en los registros seleccionados y se confirma

```

compute_stats.stats_done () at stats_scalar.asm:152
152 pop r15
(gdb) nexti
compute_stats.stats_done () at stats_scalar.asm:153
153 pop r14
(gdb) nexti
compute_stats.stats_done () at stats_scalar.asm:154
154 pop r13
(gdb) nexti
compute_stats.stats_done () at stats_scalar.asm:155
155 pop r12
(gdb) nexti
compute_stats.stats_done () at stats_scalar.asm:156
156 pop rbp
(gdb) nexti
compute_stats.stats_done () at stats_scalar.asm:157
157 pop rbx
(gdb) nexti
compute_stats.stats_done () at stats_scalar.asm:159
159 ret
(gdb) info registers r13 r14 r15 rbp
r13 0x7ffff7ffdd00 140737354125312
r14 0x7ffff7ffdd50 1407373488346448
r15 0x555555555555 93824992247240
rbp 0x7ffff7ffdc00 0x7ffff7ffdc00
(gdb) |

```

Figura 20: Stats_done del GDB compute_stats escalar

7.1.3. GDB en Normalize_array Escalar

Este segmento define la sección de código dada por

Listing 21: Código normalize_array escalar

normalize_array :

```

movss    xmm8, xmm0                ; xmm8 = mean
movss    xmm9, xmm1                ; xmm9 = stddev
xorps    xmm7, xmm7                ; xmm7 = 0.0
ucomiss  xmm9, xmm7
je      .normalize_copy
xor     eax, eax                  ; i = 0

```

.normalize_loop:

```

cmp     eax, edx                  ; i >= n?
jge     .normalize_done
movss    xmm2, [rdi + rax*4]        ; xmm2 = in[i]
subss    xmm2, xmm8                ; xmm2 = in[i] - mean
divss    xmm2, xmm9                ; xmm2 = (in[i] - mean) / stddev
movss    [rsi + rax*4], xmm2        ; out[i] = resultado

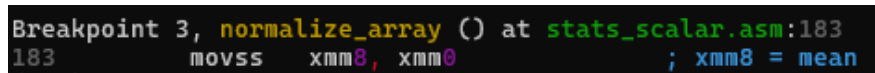
inc     eax                      ; i++
jmp     .normalize_loop
.
.
.
.

```

O visto de manera más completa en la Figura 7

Ya que su funcionalidad se definió en la sección del Normalize_array escalar, solamente se va a comprobar su funcionamiento

El código ya está activo para este punto, pero si se vuelve a correr con el mismo comando *runinput.datoutput.dat100*, entonces va a saltar hasta el siguiente break point como se aprecia en la Figura 21



```

Breakpoint 3, normalize_array () at stats_scalar.asm:183
183      movss    xmm8, xmm0          ; xmm8 = mean

```

Figura 21: Primera línea del GDB Normalize_array escalar

Este proceso lo que hace es que busca guardar el acumulado del valor normalizado en XMM2, de la forma en la que se propuso anteriormente. tras varias iteraciones del proceso se tiene como resultado la Figura 22

```

104      jge     .normalize_done
(gdb)
105      movss  xmm2, [rdi + rax*4] ; xmm2 = in[i]
(gdb)
106      subss  xmm2, xmm3         ; xmm2 = in[i] - mean
(gdb)
107      divss  xmm2, xmm3         ; xmm2 = (in[i] - mean) / stddev
(gdb)
108      movss  [rsi + rax*4], xmm2 ; out[i] = resultado
(gdb)
109      inc    eax                ; i++
(gdb)
110      jmp    .normalize_loop
(gdb)
111      cmp    eax, edx           ; i >= n?
(gdb)
112      jge    .normalize_done
(gdb)
normalize_array.normalize_done () at stats_scalar.asm:228
228      ret
(gdb) info registers eax edx
eax      0x18      16
edx      0x18      16
(gdb) info registers xmm2
xmm2     {v0_half = {0x40d5, 0x3fd0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v0_float = {0x3fd040d5, 0x0, 0x0, 0x0}, v2_double = {0x3fd040d5, 0x0}, v6_int8 = {0xd5, 0x40, 0xd0, 0x2f, 0x0 <repeats 12 times>}, v8_int16 = {0x40d5, 0x3fd0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v4_int32 = {0x3fd040d5, 0x0, 0x0, 0x0}, v2_int64 = {0x3fd040d5, 0x0}, uint128 = 0x3fd040d5}
(gdb)

```

Figura 22: Final del GDB Normalize_array escalar

Que como se puede apreciar contiene el valor normalizado final

7.2. Evidencia de GDB vectorial

Para la sesion con GDB se presenta la ejecución vectorial con el caso $N = 16$. Este tamaño de entrada permite observar dos iteraciones completas del bucle vectorial de `sum_array`, cada una con ocho valores float. En GDB se colocaron break points en `sum_array.sum_vec_loop` y `normalize_array.norm_done`.

Al detenerse al inicio del bucle de suma, se avanzó con `ni 4`. Este comando ejecutó `cmp`, `jge`, `vmovaps` y `vaddps`. Con esto el GDB se posiciona en `add eax, 8`, donde la suma vectorial ya se aplico, pero el índice todavía no se incrementa. Y se consultaron los ocho carriles con `p $ymm1.v8_float` y `p $ymm0.v8_float`.

La Fig 23 muestra la primera iteración, `ymm1` contiene los valores cargados del arreglo y `ymm0` contiene los mismos valores porque es el acumulador.

```

Breakpoint 1, sum_array.sum_vec_loop () at asm/vector/stats_vector.asm:34
34      cmp    eax, ecx
[(gdb) display/i $pc
1: x/i $pc
=> 0x55555555aaf <sum_array.sum_vec_loop>:      cmp    %ecx,%eax
(gdb) ni 4
38      add    eax, 8
1: x/i $pc
=> 0x55555555abc <sum_array.sum_vec_loop+13>:  add    $0x8,%eax
(gdb) p $ymm1.v8_float
$1 = {-27.0333138, -21.9100685, -30.2605629, 12.054409, 28.644001, 30.7606869, -30.2904911, -1.43018961}
[(gdb) p $ymm0.v8_float
$2 = {-27.0333138, -21.9100685, -30.2605629, 12.054409, 28.644001, 30.7606869, -30.2904911, -1.43018961}
(gdb)

```

Figura 23: Registros YMM después del primer `vaddps` en `sum_array` vectorial.

En la Fig 24 se aprecia la segunda iteración. Se observa que `esi=n=16` y `eax=i=8`, el siguiente bloque comienza en `arr[8]`. Ahora cada carril de `ymm0` contiene la suma de dos elementos de ocho posiciones. Estos ocho valores todavía son sumas parciales, seguidamente se combinan en la reducción horizontal.

```

(gdb) p/d $esi
$3 = 16
(gdb) continue
Continuing.

Breakpoint 1, sum_array.sum_vec_loop () at asm/vector/stats_vector.asm:34
34      cmp     eax, ecx
1: x/i $pc
=> 0x55555555aaf <sum_array.sum_vec_loop>:    cmp     %ecx,%eax
(gdb) p/d $eax
$4 = 8
(gdb) ni 4
38      add     eax, 8
1: x/i $pc
=> 0x55555555abc <sum_array.sum_vec_loop+13>: add     $0x8,%eax
(gdb) p $ymm1.v8_float
$5 = {-6.28940964, 81.6190567, -4.53489494, 1.66569412, 21.9788876, 15.2346525, -63.9373627, 17.3357716}
(gdb) p $ymm0.v8_float
$6 = {-33.3227234, 59.7089882, -34.7954559, 13.7201033, 50.6228867, 45.9953384, -94.2278519, 15.9055824}

```

Figura 24: Acumulador ymm0 después del segundo vaddps, con $N = 16$ y $eax=8$.

Finalmente, se continuó hasta `normalize_array.norm_done`, después del bucle de normalización. En este punto $eax=edx=16$, por lo que ya se procesaron todos los elementos. Según el ABI de la función, `rsi` apunta al arreglo de salida, se consulta `x/16fw $rsi` que permite observar sus 16 valores normalizados, como se muestra en la Figura 25.

```

(gdb) disable 1
(gdb) continue
Continuing.

Breakpoint 2, normalize_array.norm_done () at asm/vector/stats_vector.asm:274
274      vzeroupper
1: x/i $pc
=> 0x55555555cb4 <normalize_array.norm_done>: vzeroupper
(gdb) p/d $eax
$7 = 16
(gdb) p/d $edx
$8 = 16
(gdb) x/8fw $rsi
0x555555559480: -0.881070614    -0.722735286    -0.980809629    0.326946288
0x555555559490: 0.83965224      0.905068994    -0.981734574    -0.0897989497
(gdb) x/16fw $rsi
0x555555559480: -0.881070614    -0.722735286    -0.980809629    0.326946288
0x555555559490: 0.83965224      0.905068994    -0.981734574    -0.0897989497
0x5555555594a0: -0.239974499    2.47686124     -0.185750723    0.00588019425
0x5555555594b0: 0.633665025     0.425232619    -2.02160025     0.490168273

```

Figura 25: Contenido del arreglo de salida al terminar `normalize_array` vectorial para $N = 16$.

8. Conclusiones

Tras el trabajo realizado en este reporte se puede concluir lo siguiente:

- El programa permitió realizar el procesamiento estadístico del arreglo mediante tres funciones principales: `sum_array`, `compute_stats` y `normalize_array`. Esta organización facilitó

la implementación de las versiones escalar y vectorial con la misma interfaz, y permitió compararlas utilizando las mismas entradas.

- La implementación AVX2 aprovechó el paralelismo SIMD de los registros YMM de 256 bits para procesar ocho valores `float` por iteración. Esto redujo la cantidad de iteraciones en los bucles vectorizables respecto a la versión escalar. Cuando el tamaño del arreglo no era múltiplo de ocho, los elementos restantes se procesaron mediante un bucle escalar.
- Las pruebas de correctitud mostraron pequeñas diferencias numéricas entre ambas versiones, compatibles con el redondeo de punto flotante y con el distinto orden de acumulación. Tanto los estadísticos como los arreglos normalizados pasaron la verificación contra la referencia dentro de la tolerancia establecida.
- Las mediciones de rendimiento mostraron una clara reducción del tiempo de ejecución con AVX2. El speedup fue aproximadamente $10,24\times$ para $N = 10^3$, $8,95\times$ para $N = 10^5$ y $8,61\times$ para $N = 10^6$. Estos resultados muestran el beneficio de procesar varios elementos por instrucción en los tamaños evaluados.
- Para $N = 5 \times 10^7$, el speedup disminuyó a $4,37\times$. Esto muestra que la ventaja de la versión vectorial cambia con el tamaño del arreglo. El tiempo necesario para leer y escribir más datos podría contribuir a esta disminución, pero no se obtuvieron contadores de hardware válidos para confirmar cuánto influyó la memoria.

9. Limitaciones observadas y trabajo futuro

Las mediciones se realizaron en WSL2, por lo que sus resultados corresponden a ese entorno. Al aumentar el tamaño del arreglo de $N = 10^6$ a $N = 5 \times 10^7$, el speedup bajó de $8,61\times$ a $4,37\times$. Dado que con muchos más datos el programa dedica más tiempo a leerlos y escribirlos en memoria. AVX2 acelera los cálculos, pero no elimina ese trabajo. Como no obtuvimos contadores de hardware válidos, no podemos confirmar cuánto influyó la memoria en esta disminución.

Se intentó utilizar `perf stat` para medir ciclos, instrucciones y fallos de caché. En WSL2 los eventos de hardware no estuvieron disponibles; también se probaron contadores virtualizados en una máquina virtual Ubuntu, sin obtener conteos válidos. Por esta razón, no se calcularon IPC ni tasas de fallos de caché. El análisis cuantitativo de rendimiento se limitó a los tiempos medidos por el programa y al speedup derivado de ellos.

Como trabajo futuro, se propone repetir los experimentos en Linux nativo sobre un equipo que permita acceder a los contadores de hardware. Esto permitiría medir IPC y fallos de caché para evaluar con mayor precisión la influencia del sistema de memoria. También sería útil repetir las mediciones bajo condiciones controladas del procesador y comparar distintas estrategias de reducción horizontal.

Referencias

Andrew Canis, Jongsok Choi, Mark Aldham, Victor Zhang, Ahmed Kammoona, Jason H. Anderson, Stephen Brown, and Tomasz Czajkowski. 2011. *LegUp: high-level synthesis for FPGA-based processor/accelerator systems*. In Proceedings of the 19th ACM/SIGDA international symposium on Field programmable gate arrays (FPGA '11). ACM, New York, NY, USA, 33-36.